

Mount Kenya University

Unlocking Infinite Possibilities

Student Name	Eddy Kering
Admission Number	BSCCS/2024/61947
Course	Bachelor of Science in Computer Science
Unit Code	BIT4139
Unit Name	Ethical Hacking and Counter measures
Lecturer	Michael Nyoro
Semester / Year	2026

Table of Contents

WEEK 6: VULNERABILITY ASSESSMENT AND SCANNING

6.1 Class Activity 1 — Vulnerability Identification (Sample Network Analysis).....	1
6.2 Class Activity 2 — Vulnerability Report Analysis.....	2
6.3 Class Activity 3 — Patch Management Discussion.....	3
Why organisations delay patching.....	3
Risks of unpatched systems.....	3
Best patch management practices.....	3
6.4 Practical Task 1 — Basic Vulnerability Scan of the Lab Network.....	3
Step 1: Discover Active Hosts.....	4
Step 2: Identify Open Ports.....	4
Step 3: Vulnerability Scanning — OpenVAS.....	5
Step 4: Web Assessment — Nikto.....	8
Step 5: Document Observations.....	9
Step 6: Recommend Security Improvements.....	9
6.5 Practical Task 2 — Vulnerability Research Assignment (Five Recent CVEs).....	10
6.6 Practical Task 3 — Organisational Vulnerability Assessment Plans.....	10
University Network.....	10
Small Business.....	10
Hospital.....	11
Financial Institution.....	12
6.7 Advanced Practical Task — Vulnerability Tracking Spreadsheet.....	12
6.8 Challenge Task — Vulnerability Management Framework Design.....	12
Asset Inventory.....	13
Scanning Schedule.....	13
Risk Assessment Process.....	13
Patch Management.....	13
Reporting Procedures.....	13
Continuous Monitoring.....	13
6.9 Consolidated Recommendations.....	13
Student Reflection — Week 6.....	14

WEEK 7: WEB APPLICATION SECURITY AND OWASP

7.1 Class Activity 1 — Vulnerability Identification Exercise.....	16
7.2 Class Activity 2 — Risk Classification.....	17
7.3 Class Activity 3 — Secure Design Discussion.....	18
Why security should be built into applications.....	18
The cost of fixing vulnerabilities late.....	18
The role of developers in cybersecurity.....	18
7.4 Practical Task 1 — OWASP Top 10 Research.....	18
A01 — Broken Access Control.....	18
A02 — Cryptographic Failures.....	19
A03 — Injection.....	19
A04 — Insecure Design.....	19
A05 — Security Misconfiguration.....	20
A06 — Vulnerable and Outdated Components.....	20
A07 — Identification and Authentication Failures.....	20
A08 — Software and Data Integrity Failures.....	21
A09 — Security Logging and Monitoring Failures.....	21
A10 — Server-Side Request Forgery (SSRF).....	21
7.5 Practical Task 2 — Web Application Assessment (DVWA).....	21
Attack Surface Mapping.....	21
SQL Injection Test.....	23
Cross-Site Scripting (XSS) Test.....	24
File Upload Test.....	24
HTTP Header Analysis (Browser Developer Tools).....	25
7.6 Practical Task 3 — Security Improvement Proposal.....	26
7.7 Advanced Practical Task — Web Security Checklist.....	27
Authentication.....	27
Authorization.....	27
Input Validation.....	28
Session Management.....	28
Encryption.....	28
Logging.....	28
Configuration Management.....	28
7.8 Class Demonstration 5 — OWASP Top 10 Case Study (Equifax 2017).....	29
7.9 Challenge Task — Secure Web Application Design Framework.....	29
1. Security Requirements.....	29
2. Threat Modeling.....	30
3. Secure Coding Practices.....	30
4. Testing Procedures.....	30
5. Deployment Controls.....	31
6. Monitoring and Incident Response.....	31
Student Reflection — Week 7.....	31

Week 6: Vulnerability Assessment and Scanning

Date	June 18, 2026
Week	Week 6
Tool(s) Used	Nmap (host discovery, port scanning, NSE vulnerability scripts), OpenVAS, Nikto, CVE / NVD Database, Microsoft Word, Microsoft Excel
Target	Class activities use a representative sample network and a sample vulnerability report; Practical Task 1 targets the local lab network subnet (multiple discovered hosts)
Objective	Complete all Week 6 class activities and practical tasks per the unit lesson: vulnerability identification, report analysis, patch management discussion, hands-on scanning, CVE research, organisational planning, vulnerability tracking, and framework design.

6.1 Class Activity 1 — Vulnerability Identification (Sample Network Analysis)

Rather than reusing a single previously-scanned lab host, this activity analyses a representative small-organisation network on paper, in line with the lesson's instruction to “analyse a sample network.” The sample network below was used as the basis for identifying and categorising vulnerabilities by severity.

Asset	Sample IP	Role	Configuration Note
Edge Router / Firewall	10.0.0.1	Perimeter security device	Still using the vendor default administrator credentials
Web Server	10.0.0.10	Hosts the company website	Running Apache 2.2 (end-of-life, unpatched)
Windows File Server	10.0.0.15	Shared document storage	SMBv1 enabled for legacy compatibility
Database Server	10.0.0.20	Stores customer records	MySQL reachable from the entire LAN, not just the app server
Employee Workstations (x15)	10.0.0.50–64	Day-to-day staff use	Several machines missing the last two months of Windows updates
Wireless Access Point	10.0.0.5	Staff and guest Wi-Fi	WPA2-Personal with a short, reused passphrase; guest network shares the staff VLAN

Vulnerability identification and severity categorisation:

Vulnerability	Affected Asset	Severity	Justification
Default administrator credentials	Edge Router / Firewall	Critical	Full perimeter compromise; attacker could reconfigure firewall rules or redirect traffic
End-of-life Apache 2.2	Web Server	High	Multiple public CVEs exist; the vendor no longer issues security patches
SMBv1 enabled	File Server	High	Vulnerable to wormable, network-propagating exploits
Database reachable from entire LAN	Database Server	Medium	Unnecessarily large attack surface; access should be restricted to the application server only
Missing OS security patches	Workstations	Medium	Known, frequently-exploited operating system vulnerabilities remain open
Weak, reused Wi-Fi passphrase	Wireless AP	Medium	Enables unauthorised network access via offline password cracking
Verbose server banners (version disclosure)	Web Server	Low	Helps an attacker fingerprint software versions but is not directly exploitable
Guest Wi-Fi shares the staff VLAN	Wireless AP	Informational	Best-practice segmentation gap with no currently known active exploit

6.2 Class Activity 2 — Vulnerability Report Analysis

The sample vulnerability report excerpt below was reviewed and analysed to answer the three questions set in the lesson.

Sample Vulnerability Report Excerpt

Vulnerability Name: Apache HTTP Server Path Traversal and Remote Code Execution

CVE Reference: CVE-2021-41773

Affected System: Apache HTTP Server 2.4.49

Severity Rating: High (CVSS 7.5); escalates toward critical-level remote code execution if CGI scripts are enabled

Description: A flaw in path normalisation allows requests to map outside the configured document root. If the default “require all denied” protection is not enforced and CGI scripts are active on the aliased path, this can lead to full remote code execution.

Recommended Fix: Upgrade to Apache 2.4.51 or later. Until patched, disable CGI handlers on the affected paths and verify that access outside the web root is explicitly denied.

Question	Answer
What vulnerability was discovered?	A path traversal flaw in Apache HTTP Server 2.4.49 caused by a path-normalisation defect, which can be escalated to remote code execution when CGI scripts are enabled on the affected path.
How severe is it?	Officially rated High (CVSS 7.5 per NVD), but several vulnerability scanners flag it as Critical in practice, because an unauthenticated attacker can fully compromise the server with no user interaction once CGI is enabled.
What remediation is recommended?	Apply the vendor patch immediately by upgrading to Apache 2.4.51 or later. As an interim measure, disable CGI handlers on the affected paths and confirm that “require all denied” is enforced outside the document root.

6.3 Class Activity 3 — Patch Management Discussion

Why organisations delay patching

Patching is often delayed because of compatibility testing requirements, fear of breaking dependent applications, limited IT staffing, and the operational cost of downtime — especially for systems that must run continuously, such as hospital or banking platforms. The sheer volume of monthly disclosures (Microsoft alone shipped over 200 CVEs in its June 2026 Patch Tuesday release) also creates patch fatigue, making it difficult for under-resourced teams to keep pace.

Risks of unpatched systems

Unpatched systems remain exposed to automated exploitation by worms, botnets, and ransomware operators who specifically scan the internet for known, unpatched CVEs. Once a foothold is gained, attackers can move laterally to other systems, exfiltrate data, or disrupt operations. Beyond the direct technical risk, organisations face regulatory penalties, financial loss, and reputational damage when a breach is later traced back to a vulnerability for which a patch had been available for months.

Best patch management practices

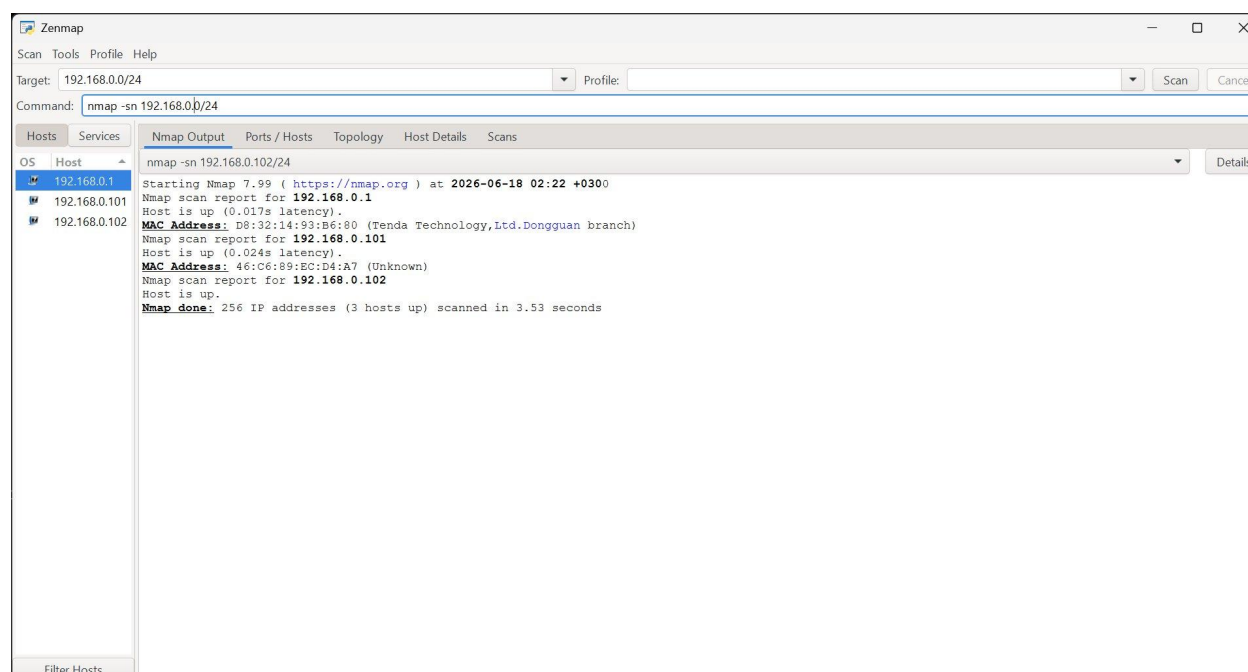
Effective patch management starts with a complete, current asset inventory so nothing is missed, followed by risk-based prioritisation that patches Critical and High severity vulnerabilities first. Patches should be tested in a staging environment before a staged rollout to production, supported by defined maintenance windows and a documented rollback plan. Automated patch management tools (e.g., WSUS, SCCM, or a cloud-based equivalent) help apply updates at scale, and systems should be re-scanned after patching to verify the fix actually closed the vulnerability.

6.4 Practical Task 1 — Basic Vulnerability Scan of the Lab Network

This task assesses the laboratory environment as a whole local network rather than a single previously-used host, so the targets here are whichever devices are found live on the lab subnet at scan time.

Step 1: Discover Active Hosts

Command: `nmap -sn 192.168.0.0/24`



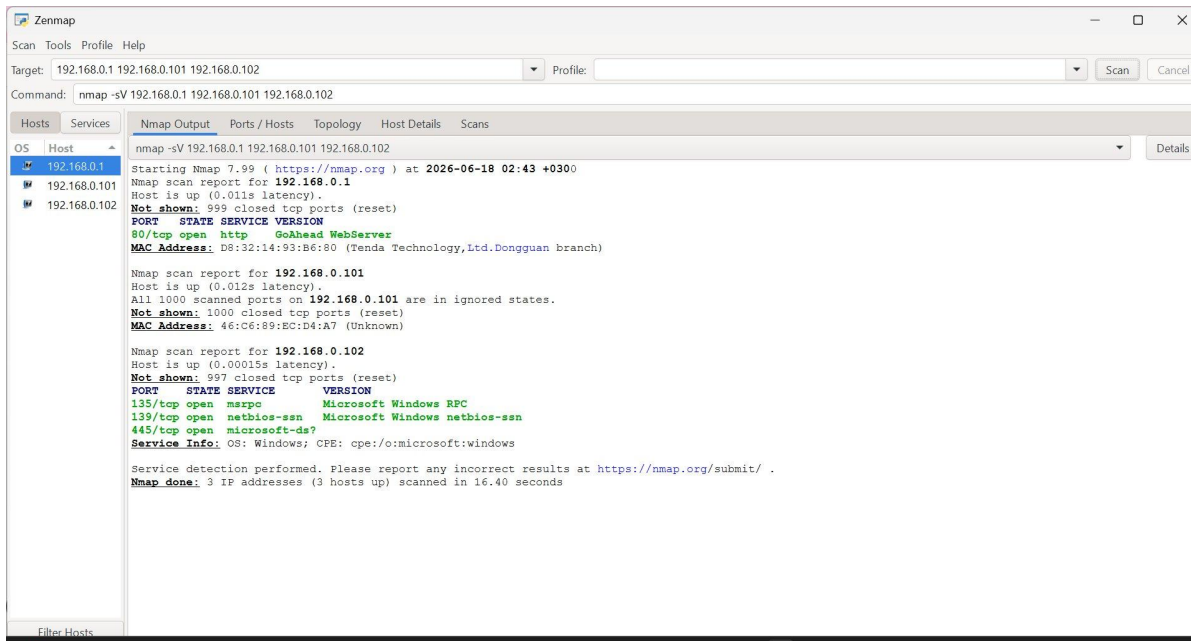
Zenmap output of: `nmap -sn 192.168.0.0/24` (host discovery / ping sweep results)

Host IP	Hostname / Device Type	Notes
192.168.0.1	Router / Gateway	MAC D8:32:14:93:B6:80, vendor identified by Nmap as Tenda Technology — this is the home router
192.168.0.101	Known host (lab target used in Weeks 1–2)	MAC 46:C6:89:EC:D4:A7, vendor unknown to Nmap
192.168.0.102	This scanning machine	No MAC reported, as expected — Nmap does not ARP-resolve its own local interface

3 hosts were found up out of the 256 addresses scanned, in 3.53 seconds.

Step 2: Identify Open Ports

Command: `nmap -sV <each discovered host>`



Terminal output of: nmap -sV (open ports and service versions for the discovered hosts)

Host	Open Ports	Services / Versions
192.168.0.1 (Router/Gateway)	80/tcp open	GoAhead WebServer (http)
192.168.0.101	None open	All 1000 scanned ports in ignored/filtered state — host up but not responsive on common ports
192.168.0.102 (scanning machine)	135/tcp, 139/tcp, 445/tcp open	135: Microsoft Windows RPC (msrpc); 139: Microsoft Windows netbios-ssn; 445: microsoft-ds

Step 3: Vulnerability Scanning — OpenVAS

OpenVAS (Greenbone Security Assistant v26.17.0) was installed on the Kali VM and successfully started. The NVT feed (95,089 NVTs) completed synchronisation; however, the SCAP, CERT, and GVMD_DATA feeds did not finish updating within the assessment window despite multiple sync attempts. Without the GVMD_DATA feed the default Scan Config is unavailable, preventing the Task Wizard from launching a scan. The Feed Status page below documents the state of the installation at the time of submission. The nmap --script vuln cross-check in Step 3b was used as the primary active vulnerability detection method and returned a confirmed real CVE (see below).

OPENVAS - Feed Status

UTC | 13:57 | admin

Type	Content	Origin	Version	Status
NVT	NVTs	Greenbone Community Feed	20260618T0839	Current
SCAP	CVEs, CPEs	Greenbone SCAP Data Feed	20260617T0643	Update in progress...
CERT	CERT-Bund Advisories, DFN-CERT Advisories	Greenbone CERT Data Feed	20260617T0553	Update in progress...
GVMD_DATA	Compliance Policies, Port Lists, Report Formats, Scan Configs	Greenbone Data Objects Feed	20260617T0645	Update in progress...

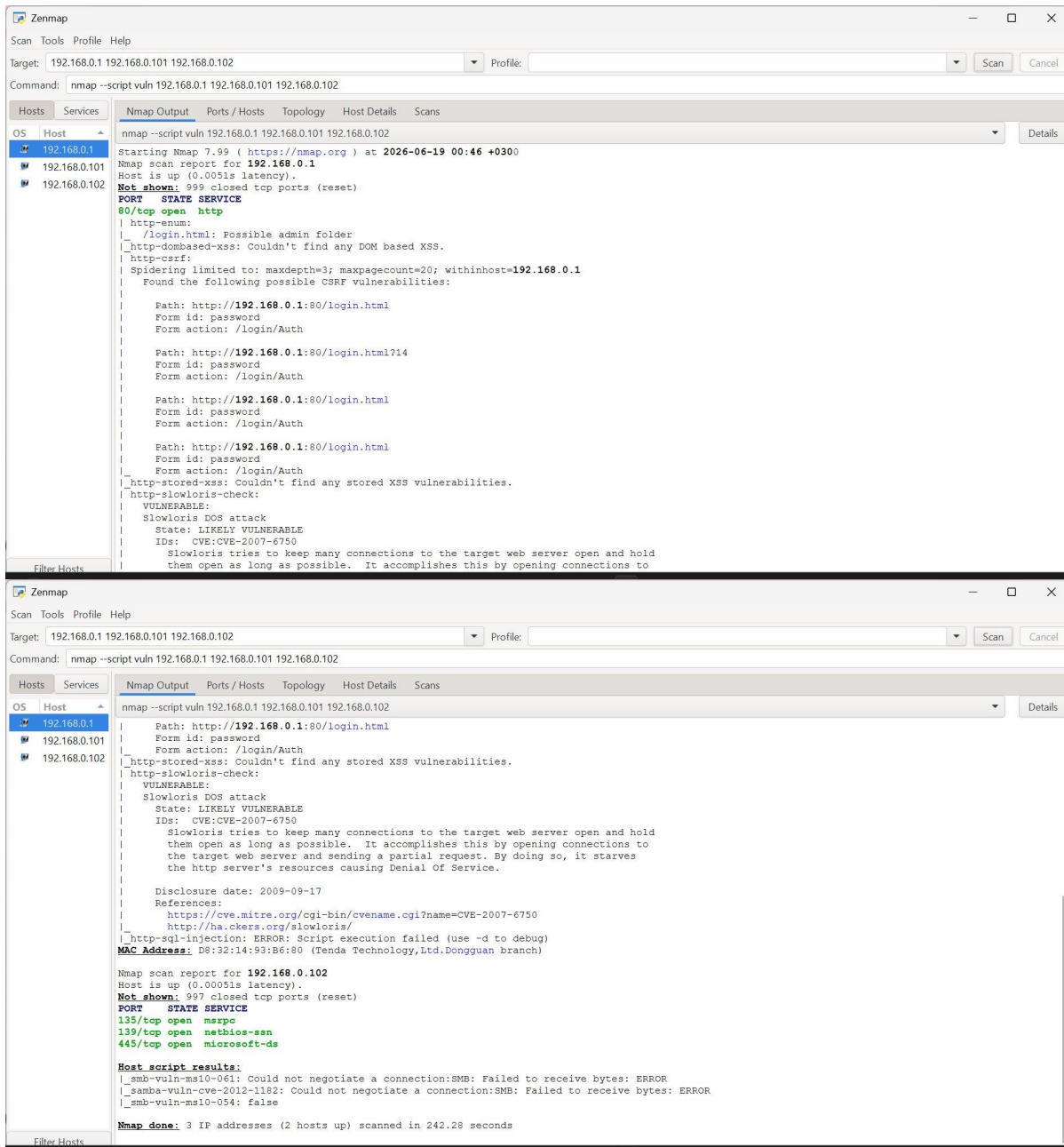
Version 26.17.0

Copyright © 2009-2026 by Greenbone AG, www.greenbone.net

OpenVAS Feed Status page showing NVT feed as Current (95,089 NVTs loaded) and SCAP / CERT / GVMD_DATA feeds with “Update in progress” — Scan Config unavailable until GVMD_DATA sync completes

As a lightweight cross-check, the Nmap NSE vuln script set was also run against the same host(s):

Command: `nmap --script vuln 192.168.0.1 192.168.0.101 192.168.0.102`



Terminal output of: `nmap --script vuln 192.168.0.1 192.168.0.101 192.168.0.102` (scrolled across two screenshots; cross-check vulnerability findings)

Host	Finding	Detail
192.168.0.1 (router)	Slowloris DoS — CVE-2007-6750	LIKELY VULNERABLE. The router's web admin interface can be starved of resources by holding many partial HTTP connections open

192.168.0.1 (router)	Possible CSRF on /login.html	The login form was flagged as a possible CSRF target during spidering — worth manual verification, as Nmap's CSRF check is heuristic
192.168.0.1 (router)	DOM-based XSS, stored XSS, SQL injection	None found by the relevant NSE scripts (http-dombased-xss, http-stored-xss, http-sql-injection)
192.168.0.101	No findings	No open ports were found on this host in Step 2, so no services were available to test
192.168.0.102 (scanning machine)	smb-vuln-ms10-061, samba-vuln-cve-2012-1182	Could not negotiate an SMB connection — the same SMBv2/v3 hardening pattern seen on the Week 1–2 Windows host
192.168.0.102 (scanning machine)	smb-vuln-ms10-054	false — not vulnerable

Step 4: Web Assessment — Nikto

Port 80 was found open on the router (192.168.0.1), running a GoAhead WebServer admin interface, so Nikto was pointed at it to check for outdated software, dangerous default files, and misconfigurations.

Command: nikto -h http://192.168.0.1

```

kali@kali: ~
└─$ nikto -h http://192.168.0.1
- Nikto v2.6.0

+ Target IP:      192.168.0.1
+ Target Hostname: 192.168.0.1
+ Target Port:    80
+ Platform:       Unknown
+ Start Time:     2026-06-18 06:08:26 (GMT-4)

+ Server: GoAhead-Webs
+ ERROR: Failed to check for updates: 403
+ No CGI Directories found (use "-C all" to force check all possible dirs). CGI tests skipped.
+ [013587] /: Suggested security header missing: referer-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referer-Policy
+ [013587] /: Suggested security header missing: x-content-type-options. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-Type-Options
+ [013587] /: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Content-Security-Policy
+ [013587] /: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security
+ [013587] /: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy
+ [006322] /login.html: Admin login page/section found.
+ [007342] /: X-Frame-Options header is deprecated and was replaced with the Content-Security-Policy HTTP header with the frame-ancestors directive. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options
+ [007352] /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ 8222 requests: 2 errors and 8 items reported on the remote host
+ End Time:     2026-06-18 06:30:09 (GMT-4) (1383 seconds)

+ 1 host(s) tested

```

Terminal output of: nikto -h http://192.168.0.1 (web server findings)

Finding	Detail
Server identified	GoAhead-Webs — a lightweight embedded web server commonly used in routers and IoT devices
Admin login page found	/login.html

Missing security headers	referrer-policy, x-content-type-options, content-security-policy, strict-transport-security, permissions-policy
Deprecated header in use	X-Frame-Options (superseded by the Content-Security-Policy frame-ancestors directive)
Scan coverage	8222 requests sent; 2 errors and 8 items reported; 1 host tested in ~22 minutes

Step 5: Document Observations

Observation	Detail
Number of active hosts discovered	3 hosts up (router + 2 other devices) out of 256 addresses scanned
Most common open service across the network	No service was common to multiple hosts — the router exposes only its HTTP admin page, while the Windows host (.102) exposes RPC/NetBIOS/SMB
Highest-severity finding (OpenVAS / Nmap vuln)	Slowloris DoS (CVE-2007-6750) on the router's web admin interface, flagged LIKELY VULNERABLE by nmap --script vuln — the only confirmed, exploitable finding across all three hosts (OpenVAS results pending feed sync)
Any unauthenticated / default-credential services found	No default credentials were tested or confirmed by these scans. However, the router's web admin login (/login.html) is reachable from the network with no apparent rate-limiting, and was not checked against default credential lists — worth following up on directly

Step 6: Recommend Security Improvements

Recommendation	Reason
Disable or restrict any service not explicitly required	Reduces the network's overall attack surface
Apply vendor patches for every High/Critical finding from OpenVAS	Closes the specific weaknesses confirmed by the scan
Segment IoT / unmanaged devices onto a separate VLAN	Limits lateral movement if one device is compromised
Re-scan after remediation to verify each fix	Confirms the vulnerability management lifecycle's verification step was completed

6.5 Practical Task 2 — Vulnerability Research Assignment

Five vulnerabilities disclosed in the weeks immediately preceding this assessment were researched using vendor advisories and the NVD, illustrating that vulnerability discovery is a continuous, real-world process and not a one-time exercise.

CVE Number	Description	Severity (CVSS)	Potential Impact	Mitigation Strategy
CVE-2026-41089	Remote code execution in Windows Netlogon, triggered by a crafted request sent to a domain controller	9.8 — Critical	Unauthenticated attacker could fully compromise a domain controller; actively exploited in the wild	Apply Microsoft's May 2026 security update on all domain controllers immediately
CVE-2026-41096	Heap-based buffer overflow in the Windows DNS Client triggered by a malicious DNS response	9.8 — Critical	Remote code execution without authentication if an attacker can spoof or intercept DNS responses	Patch all Windows endpoints; monitor for rogue DNS servers / spoofing
CVE-2026-31431 ("Copy Fail")	Privilege escalation flaw in the Linux kernel crypto subsystem (algif_aead), found via AI-assisted fuzzing	7.8 — High	Local attacker can escalate to root with a short exploit script; affects most major Linux distributions	Apply kernel security updates; disable the algif_aead module where it is not required
CVE-2026-49160 ("HTTP/2 Bomb")	Uncontrolled resource consumption in Windows HTTP.sys triggered by crafted HTTP/2 requests	7.5 — High	Unauthenticated attacker can exhaust server memory and crash IIS or any HTTP.sys-based service in seconds	Apply the June 2026 update; configure the new MaxHeadersCount registry setting
CVE-2026-50507 ("Bitskrieg")	Security feature bypass in Windows BitLocker device encryption	6.8 — Medium	Attacker with physical access to a lost or stolen device can read encrypted data	Apply the June 2026 patch; enforce pre-boot authentication (TPM + PIN) on laptops

Notably, CVE-2026-49160 was first identified by an AI-assisted vulnerability research tool (OpenAI's Codex) rather than a human researcher, reflecting a fast-growing trend toward AI-assisted vulnerability discovery across the security industry in 2026.

6.6 Practical Task 3 — Organisational Vulnerability Assessment Plans

A vulnerability assessment plan was developed for four different organisation types, reflecting how scope, tooling, schedule, and reporting needs change with context, budget, and regulatory pressure.

University Network

Element	Plan
Scope	Campus workstations and labs, Wi-Fi/BYOD network, learning management system, library systems, residence hall networks, and research servers
Tools	Nmap, OpenVAS / Nessus, Nikto (LMS and student portals), Qualys for large-scale continuous scanning
Schedule	Full network scan once per semester; critical web portals (LMS, student records) scanned monthly; new devices scanned automatically on network join
Reporting Procedure	Findings go to the campus IT Security Office; Critical/High findings escalated to the CIO within 48 hours; a semester summary report goes to university leadership

Small Business

Element	Plan
Scope	Office network, point-of-sale or e-commerce systems, cloud email/accounting services, and employee endpoints
Tools	Nmap and OpenVAS (free/open-source to fit a limited budget); Nikto for any customer-facing website
Schedule	Quarterly full scan given limited IT budget and staff, plus an extra scan after any major software or infrastructure change
Reporting Procedure	A simple written report to the business owner or IT contractor; Critical findings remediated immediately, others tracked in a basic spreadsheet

Hospital

Element	Plan
Scope	Clinical and administrative networks, the electronic health record (EHR) system, networked medical devices, and guest Wi-Fi (segmented from clinical systems)
Tools	Nmap (low-impact scan profiles near medical devices), OpenVAS / Nessus with healthcare-aware scan templates, Qualys for continuous compliance monitoring
Schedule	Continuous/automated scanning of administrative systems; medical-device segments scanned only during scheduled maintenance windows to avoid disrupting patient care; full assessment at least quarterly
Reporting Procedure	Findings reported to the hospital's CISO and the biomedical engineering team for device-related issues; documented for regulatory and compliance audits

Financial Institution

Element	Plan
Scope	Core banking systems, online/mobile banking applications, the internal corporate network, ATM network, and third-party vendor connections
Tools	Nessus / Qualys for enterprise, audit-ready reporting; Burp Suite / OWASP ZAP for the online banking application; Nmap for internal network mapping
Schedule	Continuous automated vulnerability scanning; a full assessment/penetration test at least annually (often a regulatory requirement); re-scanning after every major application release
Reporting Procedure	A formal report to the Risk and Compliance department and senior management; Critical findings reported immediately per regulatory incident-reporting requirements; audit trail retained for examiner review

6.7 Advanced Practical Task — Vulnerability Tracking Spreadsheet

A vulnerability tracking spreadsheet was created (Week6_Vulnerability_Tracker.xlsx, submitted alongside this report) containing the columns specified in the lesson — Vulnerability Name, Risk Level, Status, Assigned Personnel, and Resolution Date — plus a CVE reference, identification date, an auto-calculated “Days Open” column, and notes. It carries forward real vulnerabilities already confirmed in Weeks 4–5, and leaves template rows ready for this week’s new Practical Task 1 findings, simulating how a real vulnerability management process tracks issues over time rather than treating each scan as a one-off event.

Vulnerability Name	Risk Level	Status	Resolution Date
vsftpd 2.3.4 Backdoor (CVE-2011-2523)	Critical	Resolved	05-Jun-2026
Samba username map script RCE (CVE-2007-2447)	Medium	Resolved	05-Jun-2026
distcc Daemon RCE (CVE-2004-2687)	High	Resolved	05-Jun-2026
SSLv3 Enabled — POODLE (CVE-2014-3566)	Medium	In Progress	—
World-writable SMB ‘tmp’ share	High	Open	—
SNMP default community string ‘public’	Medium	Open	—
[New Week 6 finding — see Section 6.4]	—	Open	—

6.8 Challenge Task — Vulnerability Management Framework Design

A Vulnerability Management Framework was designed covering the six components specified in the lesson, intended to apply across any of the four organisation types discussed in Section 6.6.

Component	Description / Approach
Asset Inventory	Maintain a continuously updated inventory of all hardware, software, and cloud assets (e.g., via a CMDB or asset-management tool), tagged by owner, criticality, and data sensitivity — assets that aren't inventoried can't be assessed
Scanning Schedule	Tiered cadence by asset criticality: internet-facing/critical systems scanned weekly, internal systems monthly, a full enterprise scan quarterly, plus ad-hoc scans after major changes
Risk Assessment Process	Every discovered vulnerability is scored with CVSS, then adjusted for business context (exposure, data sensitivity, compensating controls) to produce a final priority rating rather than relying on the raw base score alone
Patch Management	Risk-based remediation SLAs — e.g., Critical within 48 hours, High within 7 days, Medium within 30 days, Low at the next maintenance window — with a documented test → pilot → production rollout and rollback plan
Reporting Procedures	Two report tiers: a detailed technical report for IT/security teams, and an executive summary for leadership covering risk trends, top priorities, and KPIs
Continuous Monitoring	Beyond periodic scans, use continuous/agent-based tools and subscribe to vendor advisories and the NVD/CVE feed so newly disclosed vulnerabilities affecting the asset inventory are flagged immediately rather than waiting for the next scheduled scan

6.9 Consolidated Recommendations

Recommendation	Rationale
Replace all default credentials on network devices	Eliminates one of the most commonly exploited, easily preventable weaknesses (Class Activity 1)
Retire or upgrade end-of-life software (e.g., Apache 2.2, SMBv1)	Removes exposure to vulnerabilities that will never receive vendor patches
Adopt risk-based patch SLAs tied to CVSS severity	Ensures Critical/High findings are remediated fastest, as outlined in the Challenge Task framework
Use the vulnerability tracker to manage findings to closure	Prevents discovered issues (e.g., the open SMB share and SNMP findings) from being identified once and then forgotten
Re-scan after every remediation	Completes the vulnerability management lifecycle's verification step and confirms the fix actually worked

Student Reflection — Week 6

Reflection Question	Answer
What is a vulnerability?	A weakness in a system, application, network, or process that an attacker could exploit — for example, a weak password, outdated software, or a misconfigured server.
What is vulnerability assessment?	A systematic process of identifying, classifying, and prioritising vulnerabilities within a system in order to discover weaknesses, evaluate risk, and recommend fixes.
How does vulnerability assessment differ from penetration testing?	Vulnerability assessment is broad and largely automated, and reports weaknesses; penetration testing is more focused and manual, and demonstrates real-world impact by actually exploiting them.
Why is risk prioritisation important?	Remediation resources are always limited, so ranking vulnerabilities by severity, exploitability, and business impact ensures the most dangerous issues are fixed first.
What is CVE?	Common Vulnerabilities and Exposures — a unique public identifier assigned to a known vulnerability, e.g. CVE-2026-41089.
What is CVSS?	The Common Vulnerability Scoring System — a standardised 0.0–10.0 score used to rate vulnerability severity as Low, Medium, High, or Critical.
Why is patch management critical?	Most real-world breaches exploit vulnerabilities for which a patch already existed; timely patching closes that window of exposure before attackers can use it.
What role do vulnerability scanners play in cybersecurity?	Tools like Nmap, OpenVAS, and Nikto automate the discovery of known weaknesses at scale, feeding the risk assessment and remediation stages of the vulnerability management lifecycle.

Working through every activity this week — from the paper-based sample-network exercise to the hands-on OpenVAS/Nikto scan and the organisational planning tasks — made it clear that vulnerability assessment is as much about process and prioritisation as it is about running a tool. The same CVSS-based thinking used to rank the sample network's findings in Class Activity 1 carried through to scoring this week's lab discoveries and building the tracking spreadsheet, which reinforced why the lecture frames vulnerability management as a continuous lifecycle rather than a single scan.

Week 7: Web Application Security and OWASP

Date	June 25, 2026
Week	Week 7
Tool(s) Used	Burp Suite, OWASP ZAP, Nikto, Browser Developer Tools, DVWA (Damn Vulnerable Web Application)
Target	DVWA (local lab instance) — a deliberately vulnerable web application designed for legal security testing
Objective	Understand and demonstrate OWASP Top 10 web application vulnerabilities, perform basic web application security testing, and develop a secure design framework.

7.1 Class Activity 1 — Vulnerability Identification Exercise

A sample web application (DVWA) was reviewed to identify authentication weaknesses, input validation flaws, and configuration issues. The table below summarises findings categorised by vulnerability type.

Vulnerability Type	Finding	Location in App	Severity
Authentication Weakness	Default credentials (admin/password) accepted on login	Login page	Critical
Authentication Weakness	No account lockout after repeated failed logins	Login page	High
Input Validation Flaw	SQL Injection possible in the User ID input field	SQLi module	Critical
Input Validation Flaw	Reflected XSS via name input field — script tags not sanitised	XSS (Reflected) module	High
Input Validation Flaw	File upload accepts any file type including .php scripts	File Upload module	Critical
Configuration Issue	DVWA security level set to Low by default	DVWA Security settings	High
Configuration Issue	PHP version and server banner disclosed in HTTP response headers	All pages	Medium
Configuration Issue	Directory listing enabled on the web server	Server config	Medium

7.2 Class Activity 2 — Risk Classification

Each finding from Activity 1 was classified by severity using CVSS guidance and the OWASP risk rating methodology, with justification for each classification.

Finding	Classification	Justification
Default credentials accepted	Critical	Any unauthenticated attacker can immediately gain full admin access with zero effort
No account lockout	High	Enables automated brute-force attacks against any user account
SQL Injection in User ID field	Critical	An unauthenticated attacker can dump the entire database, bypass authentication, or in some configurations execute OS commands
Reflected XSS in name field	High	Allows an attacker to steal session cookies, redirect users, or perform actions on their behalf via a crafted link
Unrestricted file upload	Critical	Uploading a PHP web shell gives an attacker full remote code execution on the server
DVWA security set to Low	High	All security controls are disabled, making every module trivially exploitable
Server version disclosure	Medium	Aids attacker reconnaissance; not directly exploitable but reduces the effort required to find version-specific CVEs
Directory listing enabled	Medium	Exposes the application's file structure and may reveal sensitive configuration files

7.3 Class Activity 3 — Secure Design Discussion

Why security should be built into applications

Security that is added as an afterthought is significantly more expensive and less effective than security that is designed in from the start. A flaw in the authentication design of a web application, for example, cannot simply be patched later without redesigning core components. The OWASP principle of Secure by Design means that threat modelling, security requirements, and input validation are considered at the architecture stage rather than the testing stage.

The cost of fixing vulnerabilities late

Research consistently shows that the cost of fixing a security defect grows by roughly an order of magnitude at each stage of the development lifecycle — a design flaw costs ten times more to fix in production than in design. High-profile breaches (such as those caused by SQL injection or broken access control) frequently involve vulnerabilities that had been present for years and could have been prevented with basic input validation or parameterised queries from day one.

The role of developers in cybersecurity

Developers write the code that creates or eliminates vulnerabilities. A developer who understands OWASP Top 10 risks and applies secure coding practices eliminates entire vulnerability classes before they ever reach a test environment. Key developer responsibilities

include input validation, parameterised queries, proper session management, using maintained and patched dependencies, and never storing credentials or secrets in source code.

7.4 Practical Task 1 — OWASP Top 10 Research

All ten OWASP Top 10 (2021 edition) vulnerabilities were researched. For each one, the definition, a real-world example, its potential impact, and the key prevention measures are documented below.

A01 — Broken Access Control

Element	Detail
Definition	Users can access resources, functions, or data beyond their authorised privileges
Real-world example	In 2019, Facebook exposed millions of phone numbers by allowing unauthenticated API calls that were intended to be restricted to authenticated users only
Impact	Unauthorised data access, data modification, account takeover, privilege escalation, full system compromise
Prevention	Enforce access control server-side; deny by default; implement role-based access control (RBAC); log and alert on access control failures; never expose direct object references in URLs

A02 — Cryptographic Failures

Element	Detail
Definition	Sensitive data is exposed because it is not adequately protected by encryption, or because weak/outdated cryptographic algorithms are used
Real-world example	The 2013 Adobe breach exposed 153 million user records that had been encrypted with the same key using 3DES in ECB mode — allowing plaintext recovery by pattern analysis
Impact	Exposure of passwords, credit card numbers, health records, personal data; regulatory fines (GDPR, PCI-DSS)
Prevention	Use TLS 1.2+ for all data in transit; hash passwords with bcrypt or Argon2; never use MD5 or SHA-1 for passwords; encrypt data at rest; disable HTTP and enforce HTTPS

A03 — Injection

Element	Detail
Definition	Untrusted user input is interpreted as a command or query rather than data — most commonly SQL Injection, but also OS command injection, LDAP injection, and others

Element	Detail
Real-world example	The 2008 Heartland Payment Systems breach (134 million credit cards compromised) was initiated via SQL injection against a web form
Impact	Authentication bypass, full database dump, data deletion, in some cases remote code execution on the database server
Prevention	Use parameterised queries and prepared statements; apply input validation (allowlist); use an ORM; apply least privilege to database accounts; never build queries by string concatenation

A04 — Insecure Design

Element	Detail
Definition	Security weaknesses that originate in the design or architecture of the application rather than in its implementation code
Real-world example	A ride-sharing app that allows a driver to see a rider's precise home address in the booking data, enabling potential stalking — a business logic failure in the data model design
Impact	Vulnerabilities that cannot be patched without architectural changes; systemic weaknesses across entire features or workflows
Prevention	Security requirements gathered before development; threat modelling during design; security design patterns; security-focused architecture reviews; red team exercises at design stage

A05 — Security Misconfiguration

Element	Detail
Definition	Systems, servers, frameworks, or cloud services are improperly configured, leaving them unnecessarily exposed
Real-world example	The 2019 Capital One breach affected 100 million customers — caused by a misconfigured AWS WAF that allowed SSRF to the EC2 instance metadata endpoint
Impact	Unauthorised access to administrative functions, data exposure, full server compromise
Prevention	Minimal install (disable all unnecessary features, services, accounts); hardened baselines; automated configuration audits; remove or change all default credentials; disable directory listing

A06 — Vulnerable and Outdated Components

Element	Detail
---------	--------

Element	Detail
Definition	Applications use libraries, frameworks, or other software components that contain known vulnerabilities
Real-world example	The 2017 Equifax breach (147 million records) was caused by an unpatched Apache Struts vulnerability (CVE-2017-5638) for which a patch had been available for two months
Impact	Exploitation of known CVEs directly against production systems; supply-chain attacks via compromised upstream packages
Prevention	Track all dependencies (Software Bill of Materials); subscribe to CVE advisories; use dependency-checking tools (OWASP Dependency-Check, Snyk); remove unused components; patch promptly

A07 — Identification and Authentication Failures

Element	Detail
Definition	Weaknesses in authentication mechanisms that allow attackers to assume other users' identities
Real-world example	The 2012 LinkedIn breach exposed 6.5 million (later confirmed as 117 million) password hashes stored using unsalted SHA-1, which were cracked within hours
Impact	Account takeover, identity theft, credential stuffing attacks propagating across multiple platforms
Prevention	Multi-Factor Authentication (MFA); ban common/breached passwords; implement account lockout; use secure session tokens; hash passwords with bcrypt/scrypt/Argon2; invalidate sessions on logout

A08 — Software and Data Integrity Failures

Element	Detail
Definition	Code and software update pipelines that do not verify integrity, allowing malicious code to be injected into the software supply chain
Real-world example	The 2020 SolarWinds supply-chain attack injected malicious code into a signed software update delivered to ~18,000 organisations, including US government agencies
Impact	Malicious code shipped to all customers; backdoors in production; full system compromise at scale
Prevention	Code signing for all software and updates; integrity verification before installation; secure CI/CD pipelines; review and audit third-party library dependencies; use trusted repositories

A09 — Security Logging and Monitoring Failures

Element	Detail
Definition	Insufficient logging, monitoring, and alerting that allows attacks to go undetected or unresponded to for extended periods
Real-world example	The 2013 Target breach — attackers were inside the network for two weeks before external researchers (not Target's own monitoring) detected and reported the intrusion
Impact	Attacks go undetected; breaches persist for months; forensic investigation is impossible without logs; regulatory non-compliance
Prevention	Log all authentication events, access control failures, and input validation failures; centralise logs in a SIEM; define alert thresholds; test incident response procedures regularly

A10 — Server-Side Request Forgery (SSRF)

Element	Detail
Definition	The server is tricked by attacker-controlled input into making HTTP requests to unintended destinations — often internal systems or cloud metadata services
Real-world example	The 2019 Capital One breach leveraged SSRF to hit the AWS instance metadata endpoint (169.254.169.254) and retrieve IAM credentials that were then used to exfiltrate data from S3
Impact	Access to internal services behind the firewall; cloud credential theft; internal network scanning; data exfiltration
Prevention	Validate and sanitise all user-supplied URLs; allowlist permitted destinations; block access to internal/metadata IP ranges (169.254.x.x, 10.x.x.x); segment internal services from web tier

7.5 Practical Task 2 — Web Application Assessment (DVWA)

DVWA (Damn Vulnerable Web Application) was used as the legal lab target, running locally. The assessment covered the attack surface mapping, identification of forms and authentication mechanisms, and testing of key vulnerability categories.

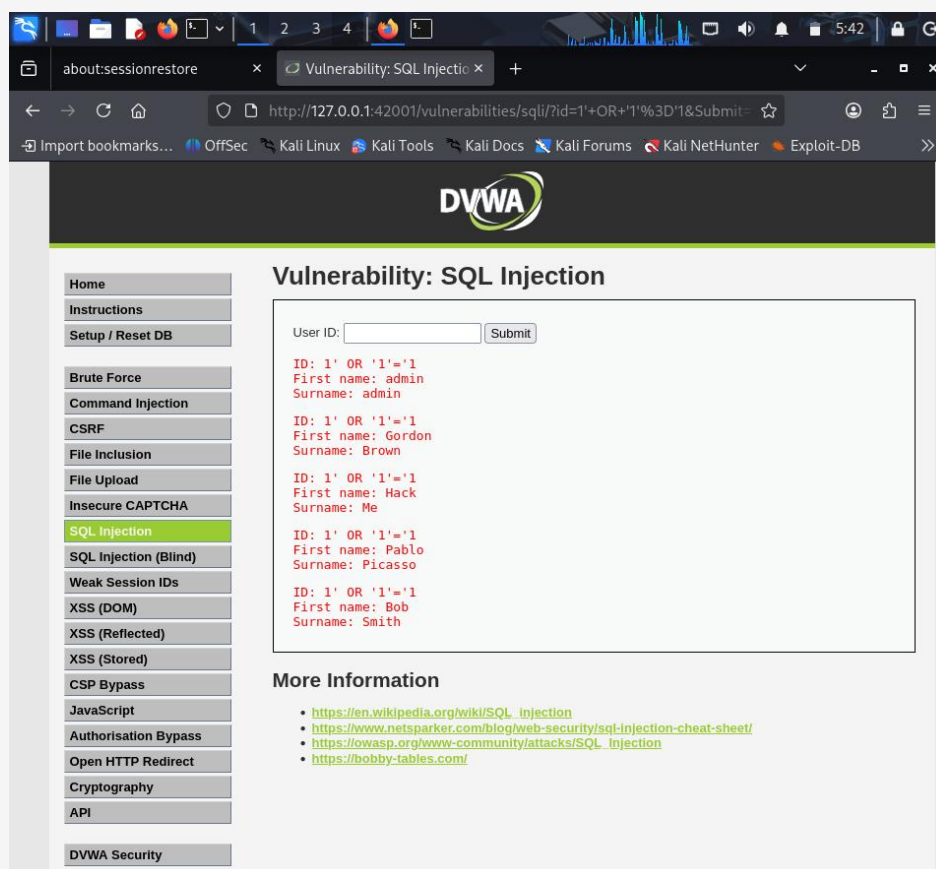
Attack Surface Mapping

Component	Description	Security Observation
Login page	Username and password form	Accepts default credentials admin/password; no CAPTCHA; no lockout
SQL Injection module	User ID lookup form	Directly interpolates user input into SQL query at Low security level
XSS (Reflected) module	Name input form	Input reflected in page without encoding; script tags execute

Component	Description	Security Observation
XSS (Stored) module	Message/guestbook form	Input stored in database and rendered for all users without sanitisation
File Upload module	File upload form	No file type restriction at Low security level; .php files accepted
Command Injection module	IP ping utility	Shell metacharacters (;, &&) allow chaining arbitrary OS commands
DVWA Security page	Admin settings panel	Security level can be lowered by any authenticated user

SQL Injection Test

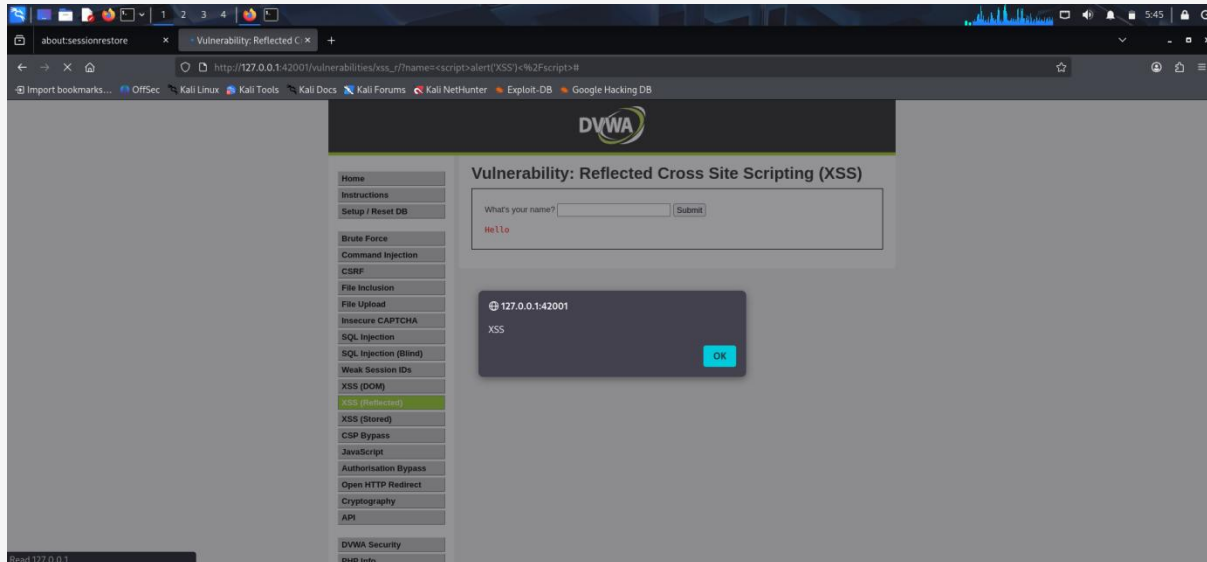
Command / Input tested: 1' OR '1'='1



DVWA SQL Injection module showing successful bypass with injected payload (or Burp Suite / ZAP intercept of the request)

Cross-Site Scripting (XSS) Test

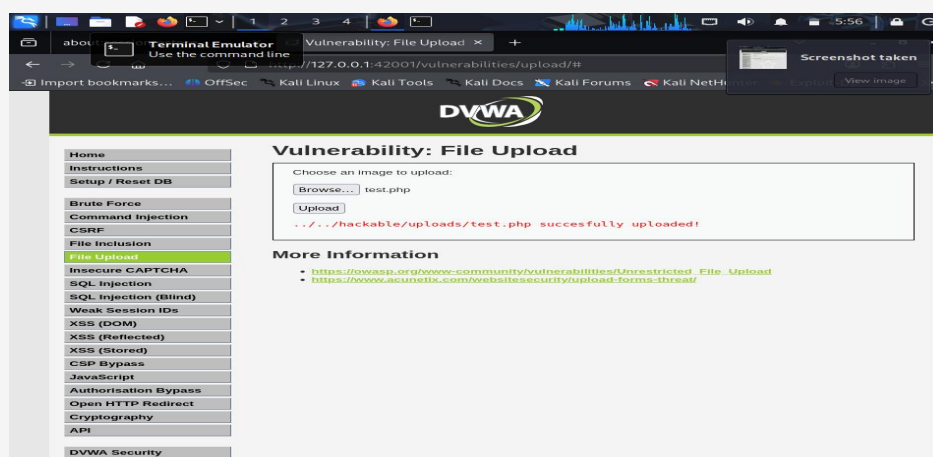
Input tested: `<script>alert('XSS')</script>`



DVWA XSS Reflected module showing the alert box triggered by the injected script

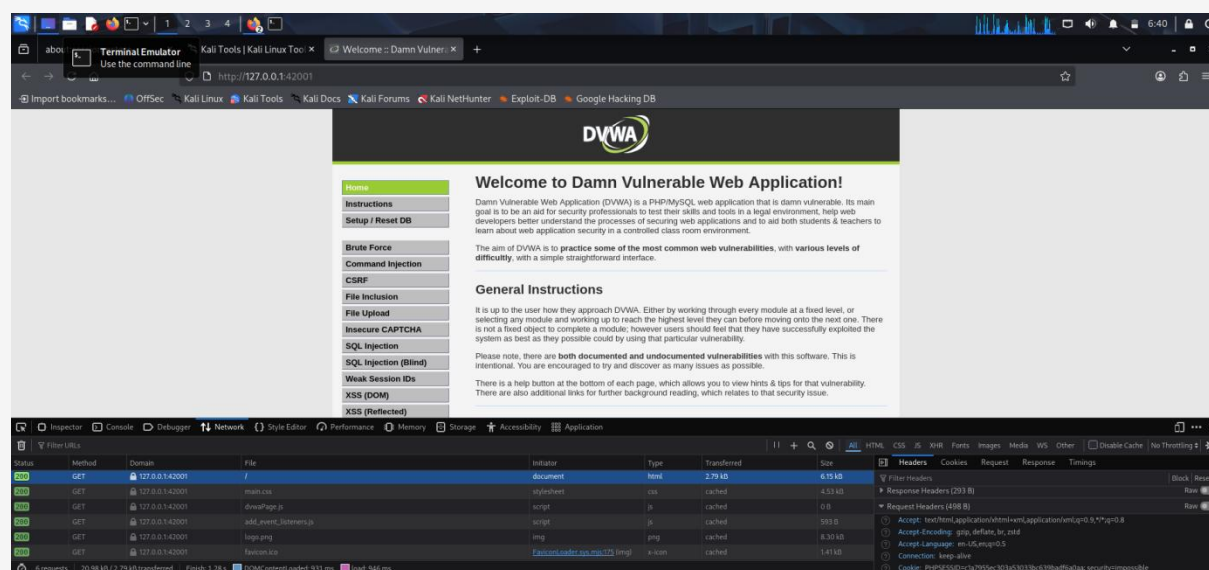
File Upload Test

A benign .php test file was uploaded to verify whether the application accepts arbitrary file types.



DVWA File Upload module showing successful upload of a .php file without restriction

HTTP Header Analysis (Browser Developer Tools)



Browser Developer Tools showing HTTP request/response headers for the DVWA login page (Network tab)

7.6 Practical Task 3 — Security Improvement Proposal

A security improvement plan was developed to remediate the vulnerabilities identified in Practical Task 2, covering authentication, input validation, logging, and patch management.

Area	Current State	Recommended Improvement
Authentication	Default credentials; no lockout; no MFA	Change all default credentials immediately; implement account lockout after 5 failed attempts; enforce MFA for all admin accounts
Input Validation	Raw user input concatenated directly into SQL queries and HTML output	Use parameterised queries for all database calls; HTML-encode all output before rendering; implement an allowlist for file upload types
Session Management	Session cookies visible in browser without Secure or HttpOnly flags	Set the Secure and HttpOnly flags on all session cookies; implement session timeout; regenerate session ID on login
File Upload	Any file type accepted at Low security level	Whitelist permitted extensions (e.g. .jpg, .png, .pdf only); validate MIME type server-side; store uploaded files outside the web root

Area	Current State	Recommended Improvement
Logging Mechanisms	No audit trail of login attempts or SQL query failures at Low security level	Log all authentication events, failed queries, and access control violations; integrate with a SIEM; alert on anomalous patterns
Patch Management	DVWA runs on PHP/Apache stack that may contain outdated components	Regularly update PHP, Apache, and all dependencies; subscribe to CVE advisories; scan for vulnerable components monthly
Configuration	Server banners disclosed; directory listing enabled	Suppress server version headers; disable directory listing; remove development tools and default files from production

7.7 Advanced Practical Task — Web Security Checklist

A comprehensive web security checklist was created covering the seven categories specified in the lesson. Each item can be used as a practical verification point during a security review or code audit.

Authentication

Check Item	Pass / Fail / N/A
Default credentials changed on all accounts	
MFA enforced for all privileged / admin accounts	
Account lockout after ≤ 5 failed login attempts	
Passwords hashed using bcrypt, scrypt, or Argon2	
Common/breached passwords blocked at registration	
Forgot-password flow uses a time-limited, single-use token	

Authorization

Check Item	Pass / Fail / N/A
Access control enforced server-side (not client-side only)	
Deny-by-default rule applied — access explicitly granted	
Direct object references not exposed in URLs	
Role-based access control (RBAC) implemented	
Horizontal privilege escalation tested (user A cannot access user B's data)	
Vertical privilege escalation tested (standard user cannot access admin functions)	

Input Validation

Check Item	Pass / Fail / N/A
All user input validated server-side (allowlist preferred)	
Parameterised queries / prepared statements used for all database calls	
HTML output encoded before rendering to prevent XSS	
File uploads restricted by extension and validated by MIME type	
File uploads stored outside the web root	
Command injection prevented — OS commands never built from user input	

Session Management

Check Item	Pass / Fail / N/A
Session tokens are long, random, and unpredictable	
Session ID regenerated after successful login	
Sessions invalidated on logout	
Session timeout enforced for inactivity	
Secure flag set on all session cookies (HTTPS only)	
HttpOnly flag set on all session cookies (inaccessible to JavaScript)	

Encryption

Check Item	Pass / Fail / N/A
HTTPS enforced site-wide; HTTP redirects to HTTPS	
TLS 1.2 or 1.3 used; SSLv3 / TLS 1.0 / 1.1 disabled	
Valid TLS certificate installed and not expired	
Sensitive data encrypted at rest	
No hardcoded secrets or credentials in source code or config files	
API keys and secrets stored in a secrets manager, not environment variables	

Logging

Check Item	Pass / Fail / N/A
All authentication events (success and failure) logged	
Access control failures logged with user context	

Check Item	Pass / Fail / N/A
Input validation failures logged	
Logs stored in a separate, append-only location	
Log data does not contain sensitive personal data (passwords, card numbers)	
Alerts configured for anomalous patterns (e.g. 100+ failed logins in 1 minute)	

Configuration Management

Check Item	Pass / Fail / N/A
Server version banners suppressed in HTTP headers	
Directory listing disabled	
Default admin accounts and passwords changed or removed	
Unnecessary services, ports, and features disabled	
Security headers set: Content-Security-Policy, X-Content-Type-Options, Strict-Transport-Security, Referrer-Policy	
All third-party components are up to date and from trusted sources	

7.8 Class Demonstration 5 — OWASP Top 10 Case Study

Case Study: The Equifax Data Breach (2017)

Element	Detail
What happened	Attackers exploited an unpatched Apache Struts vulnerability (CVE-2017-5638) in Equifax's online dispute portal to gain initial access, then moved laterally across the network for 78 days before detection
OWASP category	A06 — Vulnerable and Outdated Components (primary); A09 — Security Logging and Monitoring Failures (allowed 78-day dwell time)
Vulnerability exploited	CVE-2017-5638: A remote code execution vulnerability in Apache Struts 2, for which a patch had been available for two months before the breach
Impact	147 million consumer records exposed including Social Security numbers, birth dates, addresses, driver's licence numbers, and ~200,000 credit card numbers
Prevention measures that would have helped	Timely patch management (the patch was available); network segmentation to limit lateral movement; anomaly detection to identify 78 days of unusual data exfiltration

7.9 Challenge Task — Secure Web Application Design Framework

A Secure Web Application Design Framework was developed covering the six components specified in the lesson.

1. Security Requirements

Security requirements must be gathered and documented alongside functional requirements before development begins. These include authentication strength requirements (MFA, password policy), data classification (which fields are sensitive), access control model (who can see/edit what), regulatory compliance needs (GDPR, PCI-DSS), and encryption requirements for data in transit and at rest.

2. Threat Modeling

Using the STRIDE methodology (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege), threats are mapped to application components at the design stage. Each identified threat is rated by likelihood and impact, and a corresponding mitigation is documented before a single line of code is written.

3. Secure Coding Practices

Practice	Description
Input validation	All user input treated as untrusted; validated server-side against an allowlist before any processing
Parameterised queries	All database interactions use prepared statements or an ORM; no string concatenation in SQL
Output encoding	All user-supplied data HTML-encoded before rendering to prevent XSS
Least privilege	Each application component, database account, and API key holds only the minimum permissions required
Dependency management	Dependencies tracked in a SBOM; automatic alerts for known CVEs via tools like Snyk or Dependabot
Secrets management	No hardcoded credentials; secrets stored in a vault (e.g. HashiCorp Vault, AWS Secrets Manager)

4. Testing Procedures

Testing Type	When	Description
SAST (Static Analysis)	During development (pre-commit)	Automated code scanning for known vulnerability patterns without executing the code
DAST (Dynamic Analysis)	In staging environment	Automated web application scanning (OWASP ZAP, Burp Suite) against a running instance
Penetration Testing	Before major releases	Manual security testing by a security professional to find logic flaws and chained exploits

Testing Type	When	Description
Dependency Scanning	CI/CD pipeline	Automated scanning of third-party components against CVE databases on every build
Security Code Review	Pull request review	Manual review of code changes for security implications before merging

5. Deployment Controls

Deployments follow a hardened configuration baseline: minimal OS install, application firewall (WAF) in front of the web tier, TLS certificates validated and auto-renewed, HTTP security headers configured (CSP, HSTS, X-Content-Type-Options, Referrer-Policy), server banners suppressed, directory listing disabled, and administrative interfaces not exposed to the internet. Infrastructure is provisioned as code so that hardened configurations are applied automatically and consistently across all environments.

6. Monitoring and Incident Response

Element	Detail
Logging	All authentication events, access control failures, and input validation failures sent to a centralised SIEM
Alerting	Thresholds defined for anomalous patterns: >10 failed logins per minute, unusual data volume leaving the network, unexpected admin actions
Incident Response Plan	Defined playbooks for the most likely attack scenarios (credential compromise, XSS/injection, data exfiltration); roles and escalation paths documented
Regular drills	Tabletop exercises and simulated incidents at least quarterly to test response readiness
Post-incident review	Every security incident formally reviewed within 48 hours; root-cause analysis conducted; lessons fed back into the threat model

Student Reflection — Week 7

Reflection Question	Answer
What is web application security?	The practice of protecting web applications against attacks, unauthorised access, and data breaches — covering authentication, input validation, session management, encryption, and configuration.
Why are web applications common attack targets?	Web applications are exposed to the entire internet by design. A single flaw in a publicly accessible application can be exploited from anywhere in the world without requiring physical access or special equipment.

Reflection Question	Answer
What is OWASP?	The Open Worldwide Application Security Project — a global non-profit organisation that publishes freely available security standards, tools, and best practices, most notably the OWASP Top 10.
What is SQL Injection?	An injection attack in which an attacker inserts malicious SQL code into a user input field that is then executed by the database. This can bypass authentication, dump database contents, or in some cases execute OS commands.
What is Broken Access Control?	A category of vulnerability in which users can access data or functions beyond their intended privileges — such as viewing another user's records or accessing admin functions as a normal user.
What is SSRF?	Server-Side Request Forgery — where an attacker tricks the server into making outbound HTTP requests to unintended destinations, such as internal services or cloud metadata endpoints that would otherwise be inaccessible.
Why is secure coding important?	Most web vulnerabilities originate in the application code itself. A developer who validates input, uses parameterised queries, and applies least privilege eliminates entire vulnerability classes before they ever reach production.
How can organisations reduce web application risks?	By adopting a security-by-design approach: integrating SAST/DAST scanning into CI/CD pipelines, performing penetration testing before releases, applying the OWASP Top 10 mitigations, conducting regular dependency audits, and maintaining comprehensive logging and incident response capabilities.

This week's work reinforced how interconnected the OWASP Top 10 vulnerabilities are in practice. Testing SQL injection and XSS against DVWA made it immediately clear why parameterised queries and output encoding are so critical — the same basic failure (trusting user input) underlies multiple categories. The Equifax case study in Section 7.8 was a stark reminder that even a single unpatched component, combined with inadequate monitoring, can lead to one of the largest breaches in history. Building the Web Security Checklist in Section 7.7 also clarified how many controls need to work together — a strong password policy alone is worth little if session cookies lack the HttpOnly flag.